Unity Engineer Assessment Master Plan

Project: Senior Unity Engineer Take-Home Assessment

Repository: `C:\Assessment`

Unity target: `6000.3.10f1`

Current project version found in Step 1: `6000.4.10f1`

Generated for: detailed multi-chat execution planning

How To Use This Plan

This document is the working roadmap for the whole project. The stable step numbers are Step 0 through Step 12. In future chats, refer to a step by number, for example:

- "Continue Step 2."
- "Review Step 7 before implementation."
- "Start Step 10 and go deep."

The canonical step numbering should not change unless we explicitly agree to revise the roadmap. Substeps can be added inside a step, but the main step numbers should remain stable so that multi-chat work stays synchronized.

Project Goal

Build a production-quality Unity UI and menu system for a mock RPG game. The project is evaluated primarily on architecture, correctness, separation of concerns, reactive programming, dependency injection, memory safety, UI component coverage, responsiveness, localization, and code clarity.

The final submission should look and read like work from a senior Unity UI engineer building a shippable mobile title, not like a quick demo scene.

Non-Negotiable Architecture Rules

These rules are fixed by `AGENTS.md` and the assessment PDF.

- Strict MVC only.
- Views are MonoBehaviours only.
- Models and Controllers are pure C# classes.
- All dependencies go through Zenject / Extenject contexts.
- No singletons.
- No static mutable state.

- No `Instance` access patterns.
- No `FindObjectOfType`.
- No service locator pattern.
- No business logic in Views.
- No `Update()` polling for state changes.
- No coroutines.
- No `IEnumerator` gameplay or UI flow.
- No `StartCoroutine`.
- All timed work uses UniTask and CancellationToken.
- All reactive state uses UniRx.
- Every UniRx subscription is owned by a CompositeDisposable.
- All Addressable handles are released deterministically.
- No `Resources.Load` in app code.
- All runtime-loaded prefabs, icons, sprites, and similar assets load through Addressables.
- Every visible string uses Unity Localization.
- All text uses TextMeshPro.
- UI animation uses DOTween with easing.
- Canvas Scaler uses Scale With Screen Size at 1920x1080.
- Mobile safe area support must be implemented or clearly documented.

Canonical Step Map

Step 0: Project rules memory.

Step 1: Baseline audit.

Step 2: Required folder structure and project normalization.

Step 3: ScriptableObject configuration layer.

Step 4: ProjectContext services and cross-scene dependency layer.

Step 5: SceneContext foundations and scene wiring.

Step 6: Main Menu MVC.

Step 7: Settings Panel MVC.

Step 8: HUD Core MVC.

Step 9: HUD Advanced Systems.

Step 10: Inventory Modal MVC.

Step 11: Polish, responsiveness, localization coverage, and UX pass.

Step 12: Final audit, README, commits, and submission preparation.

Step 0 - Project Rules Memory

Objective

Create durable project instructions so future chats under this project do not lose the assessment constraints.

Current Status

Done.

Artifacts:

- `AGENTS.md`
- `README.md` pointer to `AGENTS.md`

What This Step Fixed

Step 0 created a single source of truth for:

- Required tech stack.
- Strict MVC rules.
- No-singleton and no-static-state requirements.
- Zenject ProjectContext and SceneContext expectations.
- Required folder structure.
- Screen scope.
- UI and UX standards.
- Leak-proof patterns.
- Banned-pattern audit list.
- README expectations.

Exit Criteria

Step 0 is complete when:

- `AGENTS.md` exists at the repository root.
- Future chats can read `AGENTS.md` and recover the project rules.
- The README mentions that `AGENTS.md` is the fixed project instruction source.

Future Maintenance

If we discover new assessment constraints, update `AGENTS.md` immediately. Do not let important rules live only in chat history.

Step 1 - Baseline Audit

Objective

Understand the current project state before adding code. This avoids building MVC architecture on top of broken package setup, wrong folders, dirty metadata, or misconfigured scenes.

Current Status

Done.

Artifact:

- `STEP_01_BASELINE_AUDIT.md`

Major Findings

The project is early-stage and not yet ready for implementation. Required packages are partially present, but setup is incomplete.

Highest-priority findings:

- Unity version mismatch: project is `6000.4.10f1`, assessment requires `6000.3.10f1`.
- UniTask is missing.
- Addressables package is installed, but project Addressables settings are not initialized.
- Localization package is installed, but Localization settings/tables are not initialized.
- `Assets/AdressableAssets` is misspelled.
- Required MVC folders are incomplete.
- `Assets/Scripts/Main Menu` should become `Assets/Scripts/MainMenu`.
- Gameplay scene exists but is not in Build Settings.
- Main Menu Canvas Scaler is using Constant Pixel Size at 800x600.
- Imported UniRx and Zenject examples/tests may pollute final banned-pattern scans.

How To Fix Finding 1 - Unity Version Mismatch

Current:

```
ProjectSettings/ProjectVersion.txt
m_EditorVersion: 6000.4.10f1
```

Required:

```
6000.3.10f1
```

Preferred fix:

1. Install Unity `6000.3.10f1` through Unity Hub.
2. Close the project.
3. Open the project with Unity `6000.3.10f1`.
4. Let Unity reimport.
5. Save the project.
6. Recheck `ProjectSettings/ProjectVersion.txt`.
7. Commit the resulting version/settings changes as a setup commit.

Do not manually edit `ProjectVersion.txt` as the real fix. Unity will rewrite it, and reviewers may open the project in the required version.

If `6000.3.10f1` cannot be installed:

- Keep the current version temporarily.
- Document the version deviation in README before submission.
- Treat this as a known risk.

How To Fix Finding 2 - UniTask Missing

UniTask is required for:

- Toast queue delays.
- Skill cooldown timers.
- Async Addressables loads.
- DOTween await flows.
- CancellationToken lifecycle safety.

Unity Package Manager path:

1. Open Unity.
2. Go to Window > Package Manager.
3. Press the plus button.
4. Choose Add package from git URL.
5. Add the official UniTask package URL.
6. Let Unity import and compile.
7. Verify code can reference `Cysharp.Threading.Tasks`.

Do not implement any timed operations before UniTask is installed.

How To Fix Finding 3 - MVC Folder Gaps

The final folder structure must follow the assessment. We will normalize this in Step 2 before adding code.

Required:

```
Assets/Scripts/
  App/
    Config/
    Installer/
    Persistence/
    Services/
  HUD/
    Model/
    View/
    Controller/
    Events/
    Installer/
  MainMenu/
    Model/
    View/
    Controller/
    Events/
    Installer/
  Settings/
    Model/
    View/
    Controller/
    Events/
    Installer/
  Inventory/
    Model/
    View/
    Controller/
    Events/
    Installer/
```

Best practice:

- Rename folders through Unity when possible to preserve `.meta` files.
- Because the folders are currently mostly empty, filesystem normalization is low risk, but Unity still needs to reimport afterward.

How To Fix Finding 4 - Misspelled Addressables Folder

Current incorrect folder:

```
Assets/AdressableAssets
```

Required folder:

```
Assets/AddressableAssets
```

Fix:

1. Create `Assets/AddressableAssets`.
2. Confirm whether `Assets/AdressableAssets` is empty.

3. If empty, remove the misspelled folder through Unity.
4. If anything useful exists there later, move it into the correctly spelled folder.
5. Use only `Assets/AddressableAssets` for runtime-loaded asset source content.

How To Fix Finding 5 - Addressables Not Initialized

Fix in Unity:

1. Open Window > Asset Management > Addressables > Groups.
2. If prompted, create Addressables settings.
3. Confirm `Assets/AddressableAssetsData` exists.
4. Create groups later for:

- UI icons.
- UI prefabs.
- Inventory item icons.
- Optional placeholder/minimap assets if runtime loaded.

5. Mark runtime-loaded sprites and prefabs as Addressable.
6. Load them only through the injected Addressables service.
7. Release all handles through the injected Addressables service.

How To Fix Finding 6 - Localization Not Initialized

Fix in Unity:

1. Open Edit > Project Settings > Localization.
2. Create Localization settings.
3. Add at least two locales.
4. Recommended locales:

- English `en`
- Bengali `bn` or Spanish `es`

5. Create string tables:

- Common
- MainMenu
- Settings
- HUD
- Inventory

6. Add entries for every visible string before wiring final Views.
7. Use `LocalizedString` references in config/View serialized fields.

8. Use localized TMP components or controller-fed localized values.

Important rule:

- No visible string should be hardcoded in code or inspector text fields.

How To Fix Finding 7 - Gameplay Scene Missing From Build Settings

Current Build Settings include only:

```
Assets/Scenes/Main Menu.unity
```

Fix:

1. Open File > Build Profiles or File > Build Settings.
2. Add both scenes:
- `Assets/Scenes/Main Menu.unity`
- `Assets/Scenes/In Game.unity`
3. Later decide whether to rename them:
- `MainMenu.unity`
- `Gameplay.unity`
4. If renamed, update Build Settings and scene navigation config at the same time.

Recommendation:

- Rename scenes before we implement scene navigation, not after.

How To Fix Finding 8 - Canvas Scaler Wrong

Current Main Menu scene:

```
UI Scale Mode: Constant Pixel Size
Reference Resolution: 800x600
```

Required:

```
UI Scale Mode: Scale With Screen Size
Reference Resolution: 1920x1080
```

Fix in every UI scene:

1. Select the root Canvas.
2. Set UI Scale Mode to Scale With Screen Size.
3. Set Reference Resolution to 1920x1080.
4. Set Screen Match Mode to Match Width Or Height.
5. Set Match to a balanced value such as 0.5 unless the layout proves it needs another value.

6. Add a SafeArea root RectTransform under the Canvas.
7. Put screen panels under SafeArea.
8. Test at 16:9, 18:9, and 4:3.

How To Fix Finding 9 - Third-Party Package Noise

Issue:

- UniRx Examples and Zenject OptionalExtras contain sample code that uses patterns banned for our app.
- Plugin internals may contain static state or coroutine bridges by design.

Preferred final cleanup:

1. Remove UniRx Examples if not needed.
2. Remove Zenject samples/tests/optional extras not needed by the app.
3. Keep only required runtime/editor plugin code.
4. Reopen Unity.
5. Confirm no compile errors.

If we keep the third-party code:

- Final audits must scan app-owned code separately.
- README must explain that banned-pattern rules apply to project-authored implementation code, not imported package internals.

Step 1 Exit Criteria

Step 1 is done when:

- Baseline facts are documented.
- Risks are known.
- Next step is clear.
- No implementation begins until missing setup work is acknowledged.

Step 2 - Required Folder Structure And Project Normalization

Objective

Normalize the repository structure and setup baseline before writing application code. This step prevents namespace churn, broken references, and reviewer confusion later.

Why This Step Matters

The assessment explicitly requires a folder structure. Reviewers will inspect the repository before running the project. A clean folder structure signals architectural discipline and makes the MVC separation visible before any class is opened.

Inputs

- `AGENTS.md`
- `STEP_01_BASELINE_AUDIT.md`
- Current `Assets/Scripts` folders
- Current `Assets/Scenes`
- Current package state

Detailed Work Sequence

Substep 2.1: Decide Unity version path.

- Prefer opening the project in Unity `6000.3.10f1`.
- If not immediately possible, continue structural work but keep the version mismatch visible as a risk.
- Do not pretend the mismatch is resolved until `ProjectVersion.txt` confirms it.

Substep 2.2: Install UniTask.

- Add UniTask through Unity Package Manager.
- Confirm `Cysharp.Threading.Tasks` is available.
- Do not create custom async wrappers to compensate for missing UniTask.

Substep 2.3: Normalize `Assets/Scripts`.

Create:

```
Assets/Scripts/App/Config
Assets/Scripts/App/Installer
Assets/Scripts/App/Persistence
Assets/Scripts/App/Services
Assets/Scripts/HUD/Events
Assets/Scripts/MainMenu/Model
Assets/Scripts/MainMenu/View
```

```
Assets/Scripts/MainMenu/Controller
Assets/Scripts/MainMenu/Events
Assets/Scripts/MainMenu/Installer
Assets/Scripts/Settings/Model
Assets/Scripts/Settings/View
Assets/Scripts/Settings/Controller
Assets/Scripts/Settings/Events
Assets/Scripts/Settings/Installer
Assets/Scripts/Inventory/Model
Assets/Scripts/Inventory/View
Assets/Scripts/Inventory/Controller
Assets/Scripts/Inventory/Events
Assets/Scripts/Inventory/Installer
```

Substep 2.4: Rename `Main Menu`.

Preferred:

- Rename `Assets/Scripts/Main Menu` to `Assets/Scripts/MainMenu`.

Reason:

- Folder names with spaces make namespaces, asmdef references, and scripts less clean.
- The assessment uses `MainMenu` as a feature concept.

Substep 2.5: Fix Addressable asset folder.

- Create `Assets/AddressableAssets`.
- Remove `Assets/AdressableAssets` if empty.
- If Unity warns about meta changes, handle in Unity rather than guessing.

Substep 2.6: Consider scene naming.

Current:

```
Main Menu.unity
In Game.unity
```

Recommended:

```
MainMenu.unity
Gameplay.unity
```

Do this before implementing scene navigation.

Substep 2.7: Initialize package project data.

Addressables:

- Create Addressables Settings.
- Confirm `Assets/AddressableAssetsData`.

Localization:

- Create Localization Settings.
- Add locales.
- Create first string tables.

TMP:

- Import TMP essentials.
- Confirm TMP components are available.

Substep 2.8: Create a project branch or commit.

Recommended branch:

```
codex/project-structure
```

Commit message:

```
Normalize project structure and package baseline
```

Files And Areas Touched

- `Assets/Scripts`
- `Assets/AddressableAssets`
- `Assets/Localization`
- `Assets/Scenes`
- `Assets/AddressableAssetsData`
- `Packages/manifest.json`
- `Packages/packages-lock.json`
- `ProjectSettings`

Verification

Verify:

- Required folders exist.
- Old misspelled Addressables folder is gone or intentionally empty.
- Unity opens without compile errors.
- UniTask namespace resolves.
- TMP essentials are imported.
- Addressables settings exist.
- Localization settings exist.
- Both scenes are in Build Settings if scene setup is part of this step.

Risks

- Renaming folders outside Unity can orphan `.meta` files.
- Installing UniTask may update package lock files.
- Opening with the wrong Unity version can keep rewriting project settings.

Exit Criteria

Step 2 is complete when:

- Folder structure matches `AGENTS.md`.
- UniTask is installed.
- Correct `AddressableAssets` folder exists.
- Core Unity package setup is initialized.
- No app code has been written into the wrong folder structure.

Step 3 - ScriptableObject Configuration Layer

Objective

Create all configuration ScriptableObjects before implementing controllers. This keeps magic numbers and hardcoded values out of code.

Why This Step Matters

The assessment bans magic numbers and requires ScriptableObjects for config data. If configs are created early, controllers can depend on named config interfaces rather than constants scattered through code.

Required Config Assets

Game version config:

- Path: `Assets/ScriptableObjects/App/GameVersionConfig.asset`
- Interface: `IGameVersionConfig`
- Fields:
- version string or semantic version components
- optional build label
- Used by:
- Main Menu version label

UI theme config:

- Path: `Assets/ScriptableObjects/Theme/UiThemeConfig.asset`
- Interface: `IUiThemeConfig`
- Fields:
- font family reference
- primary color
- secondary color
- warning color
- danger color
- disabled color
- panel background color
- spacing scale
- corner radius values if used
- Used by:

- all screens

Animation timing config:

- Path: `Assets/ScriptableObjects/App/UiAnimationConfig.asset`
- Interface: `IUiAnimationConfig`
- Fields:
- panel enter duration
- panel exit duration
- toast fade duration
- currency count duration
- health fill duration
- easing values
- Used by:
- Views executing DOTween animations

Scene config:

- Path: `Assets/ScriptableObjects/App/SceneConfig.asset`
- Interface: `ISceneConfig`
- Fields:
- main menu scene name
- gameplay scene name
- Used by:
- scene navigation service

Settings defaults config:

- Path: `Assets/ScriptableObjects/Settings/SettingsDefaultsConfig.asset`
- Interface: `ISettingsDefaultsConfig`
- Fields:
- default master volume
- default music volume
- default SFX volume
- default graphics quality
- default locale identifier
- Used by:
- SettingsModel
- SettingsPersistence

Graphics options config:

- Path: `Assets/ScriptableObjects/Settings/GraphicsOptionsConfig.asset`
- Interface: `IGraphicsOptionsConfig`
- Fields:
- localized option labels
- Unity quality level index mapping
- Used by:
- SettingsController
- SettingsView

Skill config:

- Path: `Assets/ScriptableObjects/HUD/SkillCatalog.asset`
- Interface: `ISkillCatalog`
- Fields per skill:
- skill id
- localized name
- Addressable icon reference
- cooldown duration
- disabled visual color or theme token
- Used by:
- HUD skill model/controller

Toast config:

- Path: `Assets/ScriptableObjects/HUD/ToastConfig.asset`
- Interface: `IToastConfig`
- Fields:
- visible duration
- enter duration
- exit duration
- max queued toasts if desired
- localized test messages if demo driven
- Used by:
- toast queue controller

HUD demo config:

- Path: `Assets/ScriptableObjects/HUD/HudDemoConfig.asset`

- Purpose:
- provide non-gameplay demo values for the assessment
- Fields:
- initial player health
- max player health
- initial boss health
- max boss health
- initial currency
- test currency increment
- Used by:
- scene installer or demo controller

Inventory item catalog:

- Path: `Assets/ScriptableObjects/Inventory/InventoryItemCatalog.asset`
- Interface: `IItemCatalog`
- Minimum:
- 6 item definitions
- Fields per item:
- item id
- localized name
- localized description
- Addressable icon reference
- stat value
- rarity
- optional category
- Used by:
- InventoryModel
- InventoryController

Inventory layout config:

- Path: `Assets/ScriptableObjects/Inventory/InventoryLayoutConfig.asset`
- Interface: `IInventoryLayoutConfig`
- Fields:
- visible slot count
- required slot count, set to 12
- columns

- cell size
- spacing
- Used by:
- InventoryView setup

Detailed Work Sequence

1. Create ScriptableObject classes in the correct feature folders.
2. Expose interfaces for configs where controllers/services depend on them.
3. Bind concrete ScriptableObject assets through ScriptableObjectInstallers or MonoInstallers.
4. Create the actual `.asset` files under `Assets/ScriptableObjects`.
5. Populate values with named, reviewable data.
6. Avoid putting visible display strings directly into config fields unless they are `LocalizedString` references.
7. Add comments only where the config behavior is not obvious.

Validation

Verify:

- No controller contains timing constants.
- No controller contains item definitions.
- No controller contains scene names as literal strings unless wrapped by config.
- No visible string is stored as plain text when it should be localized.

Exit Criteria

Step 3 is complete when:

- Every required config ScriptableObject class exists.
- Every required config asset exists.
- Controllers planned for later steps have config dependencies ready.
- ScriptableObject assets are organized by domain.

Step 4 - ProjectContext Services And Cross-Scene Dependency Layer

Objective

Build the injected service layer owned by Zenject ProjectContext. This layer wraps Unity static APIs and cross-scene services so the app does not violate dependency rules.

Why This Step Matters

Unity UI projects often leak static calls into controllers. This assessment explicitly forbids static state and hidden lookup patterns. ProjectContext lets us centralize stable services without singletons.

ProjectContext Rules

ProjectContext may bind:

- Cross-scene services.
- Configs that are stable across scenes.
- Models that must persist across scenes, such as SettingsModel.

ProjectContext must not bind:

- Scene Views.
- Scene-specific UI GameObjects.
- Scene-only controllers.
- Scene-specific model state unless deliberately persistent.

Required Services

Addressable asset service:

- Interface: `IAddressableAssetService`
- Responsibilities:
- Load Addressable sprites.
- Load Addressable prefabs.
- Instantiate Addressable prefabs.
- Track handles.
- Release handles.
- Release instances.

- Must:
- Use UniTask.
- Accept CancellationToken.
- Dispose all tracked handles.

Settings persistence:

- Interface: `ISettingsPersistence`
- Responsibilities:
- Read/write master volume.
- Read/write music volume.
- Read/write SFX volume.
- Read/write graphics quality.
- Read/write locale id.
- Implementation:
- Wrap PlayerPrefs.
- Controllers:
- Depend on interface only.

Audio mixer service:

- Interface: `IAudioMixerService`
- Responsibilities:
- Apply master/music/SFX values.
- Convert slider value to mixer dB if using AudioMixer.
- Must:
- Keep conversion logic out of Views.

Quality settings service:

- Interface: `IQualitySettingsService`
- Responsibilities:
- Apply graphics quality index.
- Wrap `QualitySettings.SetQualityLevel`.
- Must:
- Be injected into SettingsController.

Localization service:

- Interface: `ILocalizationService`
- Responsibilities:

- Expose available locales.
- Get current locale.
- Switch locale live.
- Provide completion through UniTask if initialization is async.
- Must:
- Avoid scene reload.

Scene navigator:

- Interface: `ISceneNavigator`
- Responsibilities:
- Load Main Menu scene.
- Load Gameplay scene.
- Optionally expose transition async methods.
- Implementation:
- Wrap SceneManager or ZenjectSceneLoader.
- Must:
- Use injected scene config.
- Avoid hardcoded scene names in controllers.

Application service:

- Interface: `IApplicationService`
- Responsibilities:
- Quit app.
- Handle editor-safe quit behavior if needed.
- Used by:
- MainMenuController.

Required Project Models

Settings model:

- Interface: `ISettingsModel`
- Lifetime:
- ProjectContext single.
- ReactiveProperties:
- MasterVolume
- MusicVolume

- SfxVolume
- GraphicsQuality
- SelectedLocaleId
- Reason:
- Settings may be opened from Main Menu and Gameplay.
- It should persist and restore cleanly.

Installer Structure

Recommended files:

```
Assets/Scripts/App/Installer/ProjectInstaller.cs
Assets/Scripts/App/Installer/AppConfigInstaller.cs
```

ProjectInstaller binds:

- Services.
- Persistent models.
- Interfaces to concrete classes.

AppConfigInstaller binds:

- ScriptableObject config assets.

Detailed Work Sequence

1. Create service interfaces.
2. Create service implementations.
3. Create SettingsModel.
4. Create ProjectInstaller.
5. Create config installer.
6. Create or configure Zenject ProjectContext prefab.
7. Assign installers to ProjectContext.
8. Open Unity and validate the ProjectContext installs without errors.

Validation

Verify:

- No controller directly calls PlayerPrefs.
- No controller directly calls QualitySettings.
- No controller directly calls SceneManager.

- No controller directly calls Application.Quit.
- No controller directly calls Addressables APIs except through the injected service.
- All ProjectContext-bound services are free of scene View references.

Exit Criteria

Step 4 is complete when:

- ProjectContext exists.
- ProjectInstaller exists.
- Core services compile.
- SettingsModel persists across scenes through DI.
- Services can be resolved by scene installers.

Step 5 - SceneContext Foundations And Scene Wiring

Objective

Create the scene-level Zenject and UI foundations for Main Menu and Gameplay scenes.

Why This Step Matters

SceneContexts are where scene Views enter the dependency graph. Without clean scene setup, controllers will be tempted to find Views manually, which violates the rules.

Scene Naming Decision

Recommended final scene names:

```
Assets/Scenes/MainMenu.unity
Assets/Scenes/Gameplay.unity
```

If we keep current names:

```
Assets/Scenes/Main Menu.unity
Assets/Scenes/In Game.unity
```

Then scene config must reflect those names exactly, and README should mention the naming decision only if needed.

Main Menu Scene Foundation

Required scene objects:

- SceneContext.
- EventSystem.
- Root Canvas.
- CanvasScaler.
- SafeAreaRoot.
- MainMenuRoot.
- SettingsPanelRoot.
- Optional modal overlay root.

Canvas settings:

```
Render Mode: Screen Space Overlay
UI Scale Mode: Scale With Screen Size
Reference Resolution: 1920x1080
Screen Match Mode: Match Width Or Height
Match: 0.5, adjusted only if tested
```

Installer:

```
Assets/Scripts/MainMenu/Installer/MainMenuInstaller.cs
Assets/Scripts/Settings/Installer/SettingsInstaller.cs
```

Bindings:

- MainMenuView from scene hierarchy.
- MainMenuModel as scene single.
- MainMenuController as IInitializable and IDisposable.
- SettingsView from scene hierarchy if present.
- SettingsController as IInitializable and IDisposable.

Gameplay Scene Foundation

Required scene objects:

- SceneContext.
- EventSystem.
- Root Canvas.
- CanvasScaler.
- SafeAreaRoot.
- HudRoot.
- InventoryModalRoot.
- SettingsPanelRoot if settings are available in gameplay.
- Minimap RawImage placeholder.
- Optional RenderTexture asset assigned through config or View.

Installer:

```
Assets/Scripts/HUD/Installer/HudInstaller.cs
Assets/Scripts/Inventory/Installer/InventoryInstaller.cs
Assets/Scripts/Settings/Installer/SettingsInstaller.cs
```

Bindings:

- HUD View from hierarchy.
- Inventory View from hierarchy.
- HUD models/controllers.
- Inventory models/controllers.
- Optional settings View/controller.

Safe Area Strategy

Implement a SafeAreaView MonoBehaviour:

- It is a View-level layout utility.
- It reads `Screen.safeArea`.
- It applies anchor min/max to a RectTransform.
- It should not contain business logic.
- If it uses update-like behavior, keep it restricted to layout response and avoid game state polling.

Because the assessment says no `Update()` polling for state changes, safe area layout can be event-oriented or initialization-based. A simple safe area adjustment on Awake/OnRectTransformDimensionsChange is preferred.

Detailed Work Sequence

1. Normalize scene names if chosen.
2. Add both scenes to Build Settings.
3. Create ProjectContext if Step 4 did not already create it.
4. Add SceneContext to Main Menu.
5. Add SceneContext to Gameplay.
6. Create root Canvas in each scene.
7. Configure CanvasScaler correctly.
8. Add SafeAreaRoot.
9. Add empty screen roots.
10. Add scene installers.
11. Bind placeholder Views only when the View scripts exist.
12. Open each scene and validate no Zenject install errors.

Validation

Verify:

- Both scenes are in Build Settings.
- Each scene has one SceneContext.
- SceneContext installers are assigned.
- CanvasScaler values match requirements.
- UI roots are under SafeAreaRoot.
- No scene uses `FindObjectOfType`.

Exit Criteria

Step 5 is complete when:

- Main Menu scene has working DI foundation.
- Gameplay scene has working DI foundation.
- UI root hierarchy is ready for feature screens.
- Scene lifecycle is stable.

Step 6 - Main Menu MVC

Objective

Implement the Main Menu screen using strict MVC.

Required Scope

Main Menu must include:

- Logo rendered with TMP title or static sprite.
- Play button.
- Settings button.
- Quit button.
- DOTween panel entrance animation.
- Version label driven by ScriptableObject.
- All strings through Unity Localization.

Model

Interface:

```
IMainMenuModel
```

State:

- `ReactiveProperty<bool> IsInteractable`
- `ReactiveProperty<bool> IsSettingsOpen`
- `ReactiveProperty<string> Version`

Responsibilities:

- Own menu state.
- Expose reactive state.
- Know nothing about scene loading, Unity buttons, or DOTween.

View

Interface:

```
IMainMenuView
```

View type:

- MonoBehaviour.

Serialized references:

- TMP logo or logo image.
- Play Button.
- Settings Button.
- Quit Button.
- Version TMP label or localized label target.
- Panel RectTransform.
- CanvasGroup if needed.

Observables:

- `IObservable<Unit> PlayClicked`
- `IObservable<Unit> SettingsClicked`
- `IObservable<Unit> QuitClicked`

Render methods:

- `SetInteractable(bool value)`
- `SetVersion(string value)`
- `PlayEntranceAsync(CancellationToken token)`
- `SetSettingsButtonState(bool isOpen)`

Rules:

- View does not load scenes.
- View does not quit the app.
- View does not open settings by mutating the SettingsModel.
- View does not hardcode display text.

Controller

Interface:

```
IMainMenuController
```

Lifecycle:

- Implements `IInitializable`.
- Implements `IDisposable`.

Responsibilities:

- Subscribe to View button observables.
- Subscribe to Model reactive state.
- On Play, call injected `ISceneNavigator`.
- On Settings, update menu/settings model flow.
- On Quit, call injected `IApplicationService`.
- Trigger entrance animation with cancellation.
- Dispose all subscriptions.

Dependencies:

- `IMainMenuModel`
- `IMainMenuView`
- `ISceneNavigator`
- `IApplicationService`
- `IGameVersionConfig`
- optional `ISettingsModel`

Installer

File:

```
Assets/Scripts/MainMenu/Installer/MainMenuInstaller.cs
```

Bindings:

- `IMainMenuView` from scene instance.
- `IMainMenuModel` to `MainMenuModel` as single.
- `MainMenuController` as interfaces to self.

Detailed Work Sequence

1. Create interfaces.
2. Create Model.
3. Create View MonoBehaviour.
4. Wire Button observables with UniRx.
5. Create Controller.
6. Bind through installer.
7. Add View script to scene object.

8. Assign serialized references.
9. Add localized labels.
10. Add DOTween entrance animation.
11. Test Play button navigation.
12. Test Settings button opens Settings panel.
13. Test Quit button calls app service without editor breakage.

Validation

Search project-owned code for:

- `SceneManager`
- `Application.Quit`
- `FindObjectOfType`
- `Resources.Load`
- `StartCoroutine`
- `Update(`

Only service wrappers should contain Unity API calls where needed.

Exit Criteria

Step 6 is complete when:

- Main Menu works from cold scene start.
- Buttons expose observables.
- Controller owns flow.
- Version label comes from config.
- Menu strings are localized.
- Subscriptions are disposed.

Step 7 - Settings Panel MVC

Objective

Implement the Settings panel with reactive state, PlayerPrefs persistence, quality switching, live language switching, and clean transitions.

Required Scope

Settings must include:

- Master volume slider.
- Music volume slider.
- SFX volume slider.
- Graphics dropdown with Low, Medium, High.
- Language toggle or selector with at least two languages.
- Close button.
- Values persisted through PlayerPrefs.
- Live language switching without scene reload.

Model

Interface:

```
ISettingsModel
```

Lifetime:

- ProjectContext single.

ReactiveProperties:

- `ReactiveProperty<float> MasterVolume`
- `ReactiveProperty<float> MusicVolume`
- `ReactiveProperty<float> SfxVolume`
- `ReactiveProperty<int> GraphicsQuality`
- `ReactiveProperty<string> SelectedLocaleId`
- `ReactiveProperty<bool> IsOpen`

Responsibilities:

- Own settings state.

- Expose reactive state.
- Avoid direct PlayerPrefs calls.

View

Interface:

```
ISettingsView
```

Observables:

- `IObservable<Unit> MasterVolumeChanged`
- `IObservable<Unit> MusicVolumeChanged`
- `IObservable<Unit> SfxVolumeChanged`
- `IObservable<Unit> GraphicsChanged`
- `IObservable<Unit> LanguageChanged`
- `IObservable<Unit> CloseClicked`

Read accessors:

- `float MasterVolumeValue`
- `float MusicVolumeValue`
- `float SfxVolumeValue`
- `int SelectedGraphicsIndex`
- `int SelectedLanguageIndex`

Render methods:

- `SetMasterVolume(float value, bool notify)`
- `SetMusicVolume(float value, bool notify)`
- `SetSfxVolume(float value, bool notify)`
- `SetGraphicsIndex(int index, bool notify)`
- `SetLanguageIndex(int index, bool notify)`
- `SetVisibleAsync(bool visible, CancellationToken token)`

Rules:

- View does not call PlayerPrefs.
- View does not call QualitySettings.
- View does not call LocalizationSettings directly.
- View does not decide what locale means.

Controller

Responsibilities:

- On initialize, load persisted values through `ISettingsPersistence`.
- Push loaded values into the model.
- Bind model values to View.
- Bind View interactions to model updates.
- Persist model changes.
- Apply model changes through injected services.
- Close panel through model state and View animation.

Dependencies:

- `ISettingsModel`
- `ISettingsView`
- `ISettingsPersistence`
- `IAudioMixerService`
- `IQualitySettingsService`
- `ILocalizationService`
- config interfaces

Persistence Rules

PlayerPrefs keys must be named constants in the persistence class, not repeated strings.

Example key names:

```
settings.master_volume
settings.music_volume
settings.sfx_volume
settings.graphics_quality
settings.locale
```

Only the persistence adapter should know these strings.

Detailed Work Sequence

1. Create settings interfaces.
2. Create SettingsModel if not already done in Step 4.
3. Create PlayerPrefs persistence adapter.
4. Create audio service wrapper.
5. Create quality service wrapper.

6. Create localization service wrapper.
7. Create SettingsView.
8. Create SettingsController.
9. Bind in ProjectContext and SceneContext as appropriate.
10. Set default slider values from config.
11. Save changes reactively.
12. Test close and reopen.
13. Test restart persistence.
14. Test live language switch.

Validation

Verify:

- Values restore on scene reopen.
- Values restore after editor play stop/start.
- Graphics dropdown calls quality service.
- Language switch updates visible localized UI without scene reload.
- All subscriptions are disposed.

Exit Criteria

Step 7 is complete when:

- Settings panel works in Main Menu.
- Settings state is persistent.
- Language switching is live.
- No business logic exists in View.
- No static PlayerPrefs access leaks into controllers.

Step 8 - HUD Core MVC

Objective

Build core gameplay HUD systems: player health, boss health, currency counter, and minimap placeholder.

Required Scope

HUD core must include:

- Reactive player health bar.
- Boss health bar hidden by default.
- Boss health source separate from player health.
- Animated currency counter.
- Minimap RawImage assigned a RenderTexture placeholder.

Player Health Model

Interface:

```
IPlayerHealthModel
```

ReactiveProperties:

- `ReactiveProperty<int> CurrentHealth`
- `ReactiveProperty<int> MaxHealth`
- `ReadOnlyReactiveProperty<float> NormalizedHealth`

Rules:

- Model owns health state.
- Model clamps health values.
- Model does not know about Image.fillAmount.

Boss Health Model

Interface:

```
IBossHealthModel
```

ReactiveProperties:

- `ReactiveProperty<int> CurrentHealth`

- `ReactiveProperty<int> MaxHealth`
- `ReadOnlyReactiveProperty<float> NormalizedHealth`
- `ReactiveProperty<bool> IsVisible`

Rules:

- Boss health is not stored in the player health model.
- Boss visibility is reactive.

Currency Model

Interface:

```
ICurrencyModel
```

ReactiveProperties:

- `ReactiveProperty<int> Amount`

Optional:

- `IObservable<int> DeltaStream` if the UI needs explicit delta display.

Rules:

- Model owns numeric amount.
- View animates display value only.
- Controller decides when to animate.

HUD View

Interface:

```
IHudView
```

Render methods:

- `SetPlayerHealthFill(float normalized)`
- `SetBossHealthFill(float normalized)`
- `SetBossVisibleAsync(bool visible, CancellationToken token)`
- `SetCurrencyInstant(int amount)`
- `SetCurrencyAnimatedAsync(int from, int to, CancellationToken token)`
- `SetMinimapTexture(RenderTexture texture)`

Rules:

- View touches `Image.fillAmount`.
- View touches TMP counter label.
- View touches `RawImage.texture`.
- View does not calculate health.
- View does not own currency state.

Controller

Responsibilities:

- Bind player health normalized value to View.
- Bind boss health normalized value to View.
- Bind boss visibility to View transition.
- Bind currency changes to animated counter.
- Assign minimap texture from config or scene reference.
- Dispose subscriptions.

Demo Inputs

Since no gameplay logic is required, demo changes may be driven by a pure C# demo controller or button events, but the View must not contain state-changing business logic.

Acceptable:

- A `HudDemoController` bound by Zenject for assessment demonstration.
- Debug UI buttons exposing observables to the controller.

Not acceptable:

- Health changes hardcoded inside View button methods.
- Currency logic inside MonoBehaviour View.

Detailed Work Sequence

1. Create HUD model interfaces.
2. Create player health model.
3. Create boss health model.
4. Create currency model.
5. Create HUD View interface.
6. Create HUD View MonoBehaviour.
7. Create HUD Controller.

8. Create HUD Installer.
9. Wire player health UI.
10. Wire boss health UI.
11. Wire currency counter.
12. Wire minimap RenderTexture.
13. Add demo triggers if needed.

Validation

Verify:

- Health bars react without Update polling.
- Boss bar starts hidden.
- Boss bar animates in/out.
- Currency counter animates when value changes.
- Minimap uses RawImage with RenderTexture.
- No gameplay logic lives in View.

Exit Criteria

Step 8 is complete when:

- Core HUD is visually present.
- Reactive health and currency flows work.
- Boss health source is separate.
- Minimap placeholder is correctly wired.

Step 9 - HUD Advanced Systems

Objective

Implement the advanced HUD systems: skill buttons with cooldowns and Addressable icons, plus the async notification toast queue.

Required Scope

HUD advanced systems must include:

- Three skill buttons.
- Each skill has an icon.
- Each skill has radial cooldown overlay using Image.fillAmount.
- Each skill has visually distinct disabled state.
- Icons load through Addressables.
- Toast queue supports multiple sequential notifications.
- Each toast appears for configured duration and exits.
- All timing uses UniTask and CancellationToken.

Skill Model

Interface:

```
ISkillCooldownModel
```

Per skill state:

- Skill id.
- `ReactiveProperty<float> CooldownRemaining`
- `ReactiveProperty<float> CooldownDuration`
- `ReadOnlyReactiveProperty<float> CooldownNormalized`
- `ReadOnlyReactiveProperty<bool> IsAvailable`
- Addressable icon reference from config.

Rules:

- Model owns cooldown state.
- Controller runs timer.
- View only renders overlay and disabled state.

Skill Button View

Interface:

```
ISkillButtonView
```

Observables:

```
- IObservable<Unit> Clicked
```

Render methods:

```
- SetIcon(Sprite sprite)
- SetCooldownFill(float normalized)
- SetAvailable(bool available)
```

Rules:

- View exposes click only.
- View does not start cooldown.
- View does not load Addressables.

Skill Controller

Responsibilities:

- Load each icon through `IAddressableAssetService`.
- Release icon handles when disposed or replaced.
- Subscribe to skill clicks.
- Reject clicks when skill unavailable.
- Start cooldown timer through UniTask.
- Update model cooldown state.
- Dispose all subscriptions.
- Cancel active timers on disposal.

Timer rules:

- Use UniTask.
- Use CancellationToken.
- No coroutine.
- No Update polling.

Toast Model

Interface:

```
IToastQueueModel
```

State/events:

- `Subject<ToastRequest>` or controlled enqueue method.
- Optional `ReactiveProperty<bool> IsProcessing`.

Rules:

- Model represents queue state.
- Controller owns processing loop.

Toast View

Interface:

```
IToastView
```

Methods:

- `ShowToastAsync(ToastViewData data, CancellationToken token)`

Rules:

- View animates toast in/out.
- View does not decide queue order.
- View does not delay using coroutine.

Toast Controller

Responsibilities:

- Process queued notifications sequentially.
- Await View show/hide animation.
- Delay visible duration using UniTask.
- Cancel safely on scene unload/disposal.

Detailed Work Sequence

1. Create skill config assets.
2. Create skill model classes.
3. Create skill button View class.

4. Create skill controller logic.
5. Load icons through Addressables service.
6. Render radial cooldown overlay.
7. Implement disabled state.
8. Create toast request data structure.
9. Create toast model.
10. Create toast View.
11. Create toast controller.
12. Add demo triggers for multiple toasts.
13. Test sequential queue behavior.
14. Test cancellation by changing scenes.

Validation

Verify:

- Three skills appear.
- Each icon loads through Addressables.
- Cooldown overlay fills/empties correctly.
- Button disabled state is visible.
- Rapid clicks do not break cooldown state.
- Multiple toasts play sequentially.
- Scene unload does not leave running async tasks.
- All Addressable handles are released.

Exit Criteria

Step 9 is complete when:

- Skill system is functional.
- Toast queue is functional.
- All async/timed behavior uses UniTask.
- No memory leaks are visible from subscriptions or Addressables handles.

Step 10 - Inventory Modal MVC

Objective

Implement the Inventory modal using strict MVC, Addressables-loaded icons, ScriptableObject item data, detail panel, empty slot states, and ScrollRect support.

Required Scope

Inventory must include:

- 12 slots.
- Minimum 6 items defined in ScriptableObject catalog.
- GridLayoutGroup.
- ScrollRect support.
- Empty slots show distinct visual state.
- Clicking an item shows detail panel.
- Detail panel includes icon, name, description, and stat bar.
- Item icons load at runtime through Addressables.
- No `Resources.Load`.

Item Data

ScriptableObject item fields:

- Item id.
- Localized name.
- Localized description.
- Addressable icon reference.
- Stat value.
- Rarity.
- Optional category.

Catalog:

- Contains at least 6 items.
- Used by model/controller.
- Does not live in View.

Inventory Model

Interface:

```
IInventoryModel
```

ReactiveProperties:

- `ReactiveProperty<bool> IsOpen`
- `ReactiveProperty<IReadOnlyList<InventorySlotState>> Slots`
- `ReactiveProperty<int> SelectedSlotIndex`
- `ReactiveProperty<InventoryItemState> SelectedItem`
- `ReactiveProperty<bool> IsLoadingIcons`

Rules:

- Model owns selected item state.
- Model owns slot state.
- Model does not reference Unity UI objects.
- Empty slots are explicit states, not missing objects.

Inventory View

Interface:

```
IInventoryView
```

Observables:

- `IObservable<Unit> CloseClicked`

Slot view:

```
IInventorySlotView
```

Slot observables:

- `IObservable<Unit> Clicked`

Inventory View methods:

- `BuildSlots(int count)`
- `SetSlotOccupied(int index, Sprite icon, LocalizedString name)`
- `SetSlotEmpty(int index)`
- `SetSelectedSlot(int index)`
- `SetDetail(ItemDetailViewData data)`
- `ClearDetail()`

- `SetVisibleAsync(bool visible, CancellationToken token)`

Rules:

- View handles GridLayoutGroup and ScrollRect presentation.
- View does not load item icons.
- View does not own selected item logic.
- View does not read item catalog.

Inventory Controller

Responsibilities:

- Read item catalog.
- Create 12 slot states.
- Fill at least 6 with catalog items.
- Load item icons through Addressables service.
- Push slot render data to View.
- Subscribe to slot clicks.
- Update selected item model.
- Push detail panel state to View.
- Release icon handles on close/dispose if they are not globally cached.
- Dispose all subscriptions.

Addressables Release Strategy

Option A: Controller owns loaded icon handles.

- Load icons on modal open.
- Store handles or lease objects.
- Release on modal close or controller dispose.
- Good for explicit memory safety.

Option B: Addressable asset service owns reference-counted cache.

- Controller requests icons.
- Service tracks leases.
- Controller releases leases.
- More complex, but cleaner for repeated use.

Recommended for assessment:

- Use a simple explicit service with tracked handles and controller release calls.
- Keep behavior easy to explain in interview.

Detailed Work Sequence

1. Create item ScriptableObject class.
2. Create item catalog config.
3. Create at least 6 item assets or catalog entries.
4. Mark icons Addressable.
5. Create inventory model.
6. Create inventory slot data structures.
7. Create inventory View.
8. Create inventory slot View.
9. Create slot prefab if needed.
10. Create inventory controller.
11. Bind controller/model/View in installer.
12. Load icons on open or initialize.
13. Render empty slots distinctly.
14. Wire slot clicks.
15. Render detail panel.
16. Add ScrollRect overflow support.
17. Test repeated open/close.

Validation

Verify:

- Exactly 12 slots appear.
- At least 6 item slots are populated.
- Empty slots have visible empty state.
- Detail panel updates on click.
- Icons load through Addressables.
- Handles are released.
- ScrollRect works when content overflows.
- No business logic exists in slot Views.

Exit Criteria

Step 10 is complete when:

- Inventory modal is functional.
- Item details work.
- Addressables loading/release is safe.
- Empty and occupied states are visually distinct.

Step 11 - Polish, Responsiveness, Localization Coverage, And UX Pass

Objective

Make the project feel professional and robust across required aspect ratios and interaction states.

Required UI Standards

- Canvas Scaler uses Scale With Screen Size.
- Reference resolution is 1920x1080.
- Layouts work at 16:9.
- Layouts work at 18:9.
- Layouts work at 4:3.
- Mobile safe area support is implemented or documented.
- Every interactive button has Normal, Highlighted, Pressed, and Disabled states.
- All text uses TextMeshPro.
- All visible strings are localized.
- All screens share font family.
- All screens share color palette.
- All screens share spacing system.
- Required panel transitions use DOTween easing.

Visual System Work

Theme:

- Apply central UI theme colors.
- Remove one-off colors unless justified.
- Ensure disabled, warning, and selected states are clear.

Typography:

- Use TMP only.
- Use consistent font.
- Define type hierarchy:
- screen title
- section heading

- body text
- counter text
- button label

Spacing:

- Use consistent margins.
- Use consistent slot spacing.
- Avoid overlapping elements.
- Avoid card-inside-card visual clutter.

Animation:

- Main Menu panel entrance.
- Settings panel open/close.
- Boss health in/out.
- Currency count-up.
- Toast in/out.
- Inventory modal in/out.

Every animation:

- Uses DOTween.
- Uses easing.
- Is cancellable.
- Does not run through coroutine.

Responsiveness Testing

Test view sizes:

- 1920x1080 for 16:9.
- 2160x1080 or 2340x1080 for 18:9 style wide mobile.
- 1440x1080 or 1024x768 for 4:3.

For each screen check:

- Root panel remains inside safe area.
- Buttons remain reachable.
- Text does not clip.
- Inventory grid remains usable.
- HUD does not overlap critical UI.

- Toast appears in a consistent location.
- Settings dropdown/toggle remains readable.

Localization Pass

Check all screens:

- Main Menu logo/title if text based.
- Play.
- Settings.
- Quit.
- Version prefix if any.
- Settings labels.
- Graphics options.
- Language labels.
- HUD notifications.
- Inventory item names.
- Inventory item descriptions.
- Empty slot label if visible.
- Detail panel labels.

No hardcoded display text should remain in:

- C# code.
- TMP inspector text fields.
- Button labels.
- Dropdown option text.

Detailed Work Sequence

1. Audit all Text/TMP objects.
2. Replace legacy Text if any.
3. Apply localization to every visible string.
4. Apply theme config.
5. Configure button transition states.
6. Tune DOTween timings.
7. Test aspect ratios.
8. Test safe area behavior.
9. Check repeated scene transitions.

10. Check repeated modal open/close.
11. Check no text clipping.
12. Capture screenshots if useful for README.

Validation

Verify:

- No legacy Text components.
- No hardcoded visible text.
- No layout break at required aspect ratios.
- No required transition is instant or linear.
- UI feels consistent across all screens.

Exit Criteria

Step 11 is complete when:

- The project feels visually cohesive.
- Required aspect ratios are tested.
- Localization coverage is complete.
- Button states are complete.
- Animation quality meets assessment expectations.

Step 12 - Final Audit, README, Commits, And Submission Preparation

Objective

Prepare the project for review. This step ensures the submission is architecturally clean, explainable, and easy for reviewers to run.

Final Banned-Pattern Audit

Run scans against project-owned code.

Patterns:

```
FindObjectOfType
.Instance
Resources.Load
IEnumerator
StartCoroutine
Update(
UnityEngine.UI.Text
Subscribe(
```

Expected results:

- No app code uses banned patterns.
- `Subscribe(` calls are paired with `.AddTo(_disposables)` or explicit ownership.
- Third-party plugin internals are excluded or documented.

Memory Safety Audit

Check:

- Every controller with subscriptions has CompositeDisposable.
- Every model with subscriptions has CompositeDisposable.
- Every disposable is disposed.
- Every async loop has CancellationToken.
- Every DOTween async flow is cancellable.
- Addressable handles are released.
- Addressable instances are released.
- Modal close and scene unload do not leave work running.

Architecture Audit

Check each screen:

- Model is pure C#.
- Controller is pure C#.
- View is MonoBehaviour.
- View exposes interactions as observables.
- View does not contain business logic.
- Dependencies are injected.
- Installers bind interfaces where appropriate.

README Requirements

Final README must include:

- Exact Unity version.
- How to open the project.
- How to run Main Menu scene.
- How to navigate to Gameplay.
- Required packages.
- Architecture overview.
- ProjectContext explanation.
- SceneContext explanation.
- Folder structure explanation.
- Why `Assets/Scripts/App` exists.
- Safe area approach.
- Localization setup.
- Addressables setup.
- Known limitations.
- What would be improved with more time.
- Optional AI assistance note if desired.

Commit Hygiene

Assessment warns that one giant commit is a red flag.

Recommended commit sequence:

1. Add project instructions and audit docs.
2. Normalize folder structure and package setup.

3. Add shared config ScriptableObjects.
4. Add ProjectContext services.
5. Add scene foundations.
6. Implement Main Menu MVC.
7. Implement Settings MVC.
8. Implement HUD core MVC.
9. Implement HUD advanced systems.
10. Implement Inventory MVC.
11. Polish UI and responsiveness.
12. Final README and audit cleanup.

Optional Walkthrough Video

If making a 2 to 3 minute video:

Show:

- Main Menu entrance.
- Version label.
- Settings panel.
- Volume persistence.
- Live language switch.
- Gameplay HUD.
- Health/currency changes.
- Skill cooldown.
- Toast queue.
- Boss health bar.
- Inventory modal and item details.

Keep narration architectural:

- Mention MVC separation.
- Mention Zenject injection.
- Mention UniRx subscriptions.
- Mention UniTask cancellation.
- Mention Addressables release.

Interview Preparation

Be ready to explain:

- Why ProjectContext owns services but not Views.
- Why SettingsModel is project-level.
- Why HUD models are scene-level.
- How each subscription is disposed.
- How Addressables handles are released.
- How language switching works without reload.
- Why Views expose observables.
- Why no Update polling is needed.
- How cancellation works on scene unload.
- What you would improve with more time.

Final Verification Checklist

Before submission:

- Unity opens without console errors.
- Correct Unity version is used or documented.
- Build Settings include both scenes.
- All four screens are reachable.
- Main Menu works.
- Settings works.
- HUD works.
- Inventory works.
- Localization works.
- Addressables loads work.
- Addressables release strategy is implemented.
- Aspect ratios are checked.
- README is complete.
- Git history is reasonable.

Exit Criteria

Step 12 is complete when:

- The project can be submitted as a GitHub repository.
- README explains how to run it.
- The codebase passes architecture audit.

- The UI passes scope audit.
- The implementation is ready to defend in a senior technical interview.

Screen Architecture Summary

Main Menu

Interfaces:

- `IMainMenuModel`
- `IMainMenuView`
- `IMainMenuController`

Model state:

- Interactable.
- Settings open state.
- Version string.

View events:

- Play clicked.
- Settings clicked.
- Quit clicked.

Controller decisions:

- Navigate.
- Open settings.
- Quit app.
- Trigger entrance animation.

Settings

Interfaces:

- `ISettingsModel`
- `ISettingsView`
- `ISettingsController`
- `ISettingsPersistence`
- `IAudioMixerService`
- `IQualitySettingsService`
- `ILocalizationService`

Model state:

- Master volume.
- Music volume.
- SFX volume.
- Graphics quality.
- Selected locale.
- Open state.

View events:

- Slider changed.
- Dropdown changed.
- Language changed.
- Close clicked.

Controller decisions:

- Persist values.
- Apply audio.
- Apply quality.
- Switch locale.
- Close panel.

HUD

Interfaces:

- `IPlayerHealthModel`
- `IBossHealthModel`
- `ICurrencyModel`
- `ISkillCooldownModel`
- `IToastQueueModel`
- `IHudView`
- `ISkillButtonView`
- `IToastView`

Model state:

- Player health.
- Boss health.
- Boss visibility.

- Currency.
- Skill cooldowns.
- Toast queue.

View events:

- Skill clicked.
- Optional inventory clicked.
- Optional demo buttons clicked.

Controller decisions:

- Use skill.
- Start cooldown.
- Update health render.
- Animate currency.
- Process toast queue.
- Load icons.

Inventory

Interfaces:

- `IInventoryModel`
- `IInventoryView`
- `IInventorySlotView`
- `IInventoryController`
- `IItemCatalog`

Model state:

- Open state.
- Slot list.
- Selected slot.
- Selected item.
- Loading state.

View events:

- Close clicked.
- Slot clicked.

Controller decisions:

- Populate slots.
- Load icons.
- Release icons.
- Select item.
- Show detail panel.

Enterprise Review Principles

Keep Lifetimes Obvious

If an object subscribes, loads, awaits, or owns handles, it must have a clear disposal path.

Keep Unity API Boundaries Visible

Static Unity APIs should live behind injected wrappers. Controllers should read like pure workflow code.

Keep Views Humble

Views should be visually capable but logically boring. They expose inputs and render outputs.

Keep Models Free Of Scene Objects

Models should not know about GameObjects, Images, TMP labels, Buttons, or Transforms.

Keep Config Out Of Code

Durations, colors, labels, scene names, item definitions, and cooldowns belong in config, not scattered constants.

Keep The README Honest

If something is a limitation, document it. Senior reviewers prefer honest tradeoff notes over hidden shortcuts.